前端 HTML 面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

HTML4 与 HTML5 有什么区别

HTML5 不是简单地在 HTML4 后面加了个版本号，它其实是从网页形态到能力边界的一次跃迁。

1）语义化标签成了头等大事。以前全靠 div + class 堆页面，现在有 header、nav、section、article、aside、footer 这些标签，结构清晰多了，对 SEO 和无障碍访问帮助很大。

2）原生支持多媒体。HTML4 播放视频得靠 Flash 这类插件，HTML5 直接用 `<video>` 和 `<audio>` 标签就能播放，彻底摆脱了插件依赖。

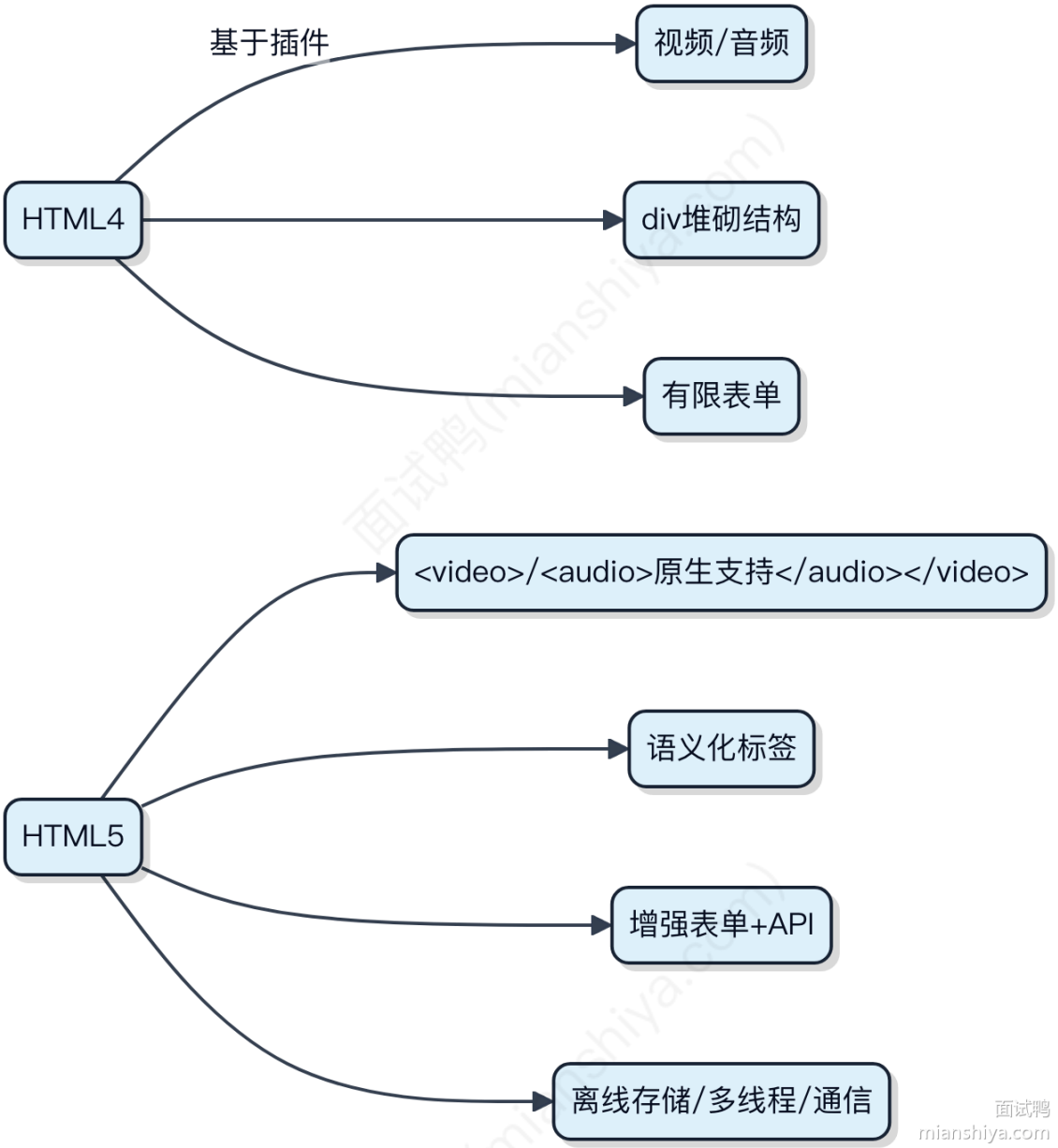
3）表单能力大幅增强。新增了 email、url、date、number、range、search 等输入类型，还有 placeholder、required、autofocus 等属性，表单验证也更友好。

4）JavaScript API 丰富了很多。比如本地存储 localStorage 和 sessionStorage，比 cookie 更方便；还有 canvas 绘图、地理定位、拖拽 API、Web Workers 多线程、WebSocket 全双工通信等，让网页能做的事接近原生应用。

5）离线能力提升。通过 Application Cache（已废弃）和后来的 Service Worker 配合 Cache API，可以实现离线访问，这是 PWA 的基础。

6）兼容性处理更明确。HTML5 容错性强，老浏览器虽然不识别新标签，但可以通过 js 创建这些元素来触发样式渲染。

面试鸭(mianshiya.com)



总的来说，HTML5 让 Web 从“看文档”变成了“用应用”。

DOCTYPE（文档类型）的作用是什么？

DOCTYPE 的作用是告诉浏览器当前 HTML 文档使用哪个版本的 HTML 标准来解析页面。没有它，浏览器可能会进入“怪异模式”，渲染行为就跟标准不一致了。

现代网页基本都用 `<!DOCTYPE html>` ，这行代码最顶上，一个字母都不能错。它不是 HTML 标签，浏览器看到它就知道接下来要用 HTML5 的规则处理文档。

比如你写个简单的页面：

```html
<!DOCTYPE html>
<html>
<head>
  <title>示例</title>
</head>
<body>
  <h1>Hello</h1>
</body>
</html>
```

只要这一行在，浏览器就按标准模式布局、盒模型、CSS 解析全都走统一规则。不然像 IE6/7 那种老时代，没 DOCTYPE 就直接怪异模式，宽度计算都能差几十像素，页面直接错乱。

现在主流框架如 React、Vue 生成的模板都默认带上这行，开发时几乎不会踩这个坑。但如果是手写静态页或者服务端直出 HTML，漏写 DOCTYPE 可能会让样式表现和预期对不上，尤其在一些嵌入式 WebKit 内核里更容易出问题。

简单说，这行就是让浏览器“规矩”干活的起点，压根不经过猜测，直接进标准模式。

HTML 的 script 标签中 defer 和 async 有什么区别？

script 标签的 defer 和 async 都能异步加载脚本，避免阻塞页面解析，但执行时机完全不同。

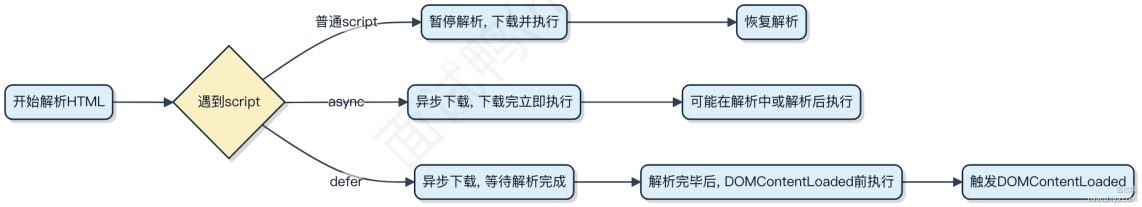
async 加载完就执行，不保证顺序，跟 script 的位置没关系。比如你写了两个 async 脚本，先写的可能后执行，完全看谁先下载完。这种适合独立脚本，像 Google Analytics 这类埋点代码，不依赖其他脚本，也不被别人依赖。

defer 则不同，它是等到 HTML 文档解析完了，DOMContentLoaded 事件触发之前按顺序执行。多个 defer 脚本会严格按照它们在文档中出现的顺序执行，适合需要操作 DOM 或有依赖关系的场景。

如果把两个都设上，现代浏览器会优先遵循标准行为：支持 defer 的情况下，async 会被忽略，表现得像只有 defer。所以别同时写，容易自己搞混。

没有 defer 或 async 的 script 会阻塞 HTML 解析，一旦遇到就停下等脚本下载执行完再继续，这也是为什么老式代码常放在 body 底部。

1）async：下载不阻塞解析，**下载完成立刻执行**，执行时会阻塞解析
2）defer：下载不阻塞解析，**等到解析完成后、DOMContentLoaded 前按序执行**



常用的 HTML meta 标签有哪些？

meta 标签是 HTML 中用于提供页面元信息的手段，不直接显示，但影响浏览器行为或被搜索引擎读取。

1）字符集声明
指定文档编码，避免乱码。现代项目基本都用 UTF-8。

```
<meta charset="UTF-8">
```

2）视口控制（viewport）
移动端适配关键，控制布局宽度和缩放。

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

没有它，移动端页面会按桌面宽度渲染，字体小得看不清。

3）页面描述与 SEO
description 被搜索引擎用作结果摘要，影响点击率。

```
<meta name="description" content="这是页面的简要说明">
```

keywords 已被主流引擎弃用，不用写。

4）HTTP 头模拟
可模拟 HTTP 响应头，比如设置刷新或缓存指令。

```
<meta http-equiv="refresh" content="5;url=https://example.com">
```

现在这类操作更多由服务端控制，前端少用。

5）Open Graph 与社交分享
让页面在微信、微博、Facebook 等平台分享时展示标题、图片等。

```
<meta property="og:title" content="分享标题">
<meta property="og:image" content="https://example.com/image.png">
```

Twitter 也有类似 meta，前缀是 `twitter:` 。

6）应用模式（iOS Safari）
让页面以全屏应用形式打开，隐藏浏览器 UI。

```
<meta name="apple-mobile-web-app-capable" content="yes">
```

这些标签看似简单，但在 SEO、移动端体验、社交传播上起着基础作用，尤其 viewport 和 og 标签，在现代网页开发中几乎是标配。

HTML5 相比于 HTML 有哪些更新？

HTML5 不是简单地在 HTML 上修修补补，而是重构了整个 Web 应用的开发模型。它让浏览器能原生支持更多功能，减少对插件的依赖。

语义化标签是重点。以前一个页面全是 div，现在可以用 header、nav、section、article、aside、footer 这些标签，结构清晰，对 SEO 和可访问性帮助很大。

多媒体支持直接内置。audio 和 video 标签出来后，不用再靠 Flash 播放音视频，像 YouTube 早期就重度依赖 Flash，现在完全可以通过原生标签加载。

表单能力增强不少。新增了 email、url、number、range、date 等输入类型，自带校验逻辑，移动端还能自动切换键盘类型，用户体验提升明显。

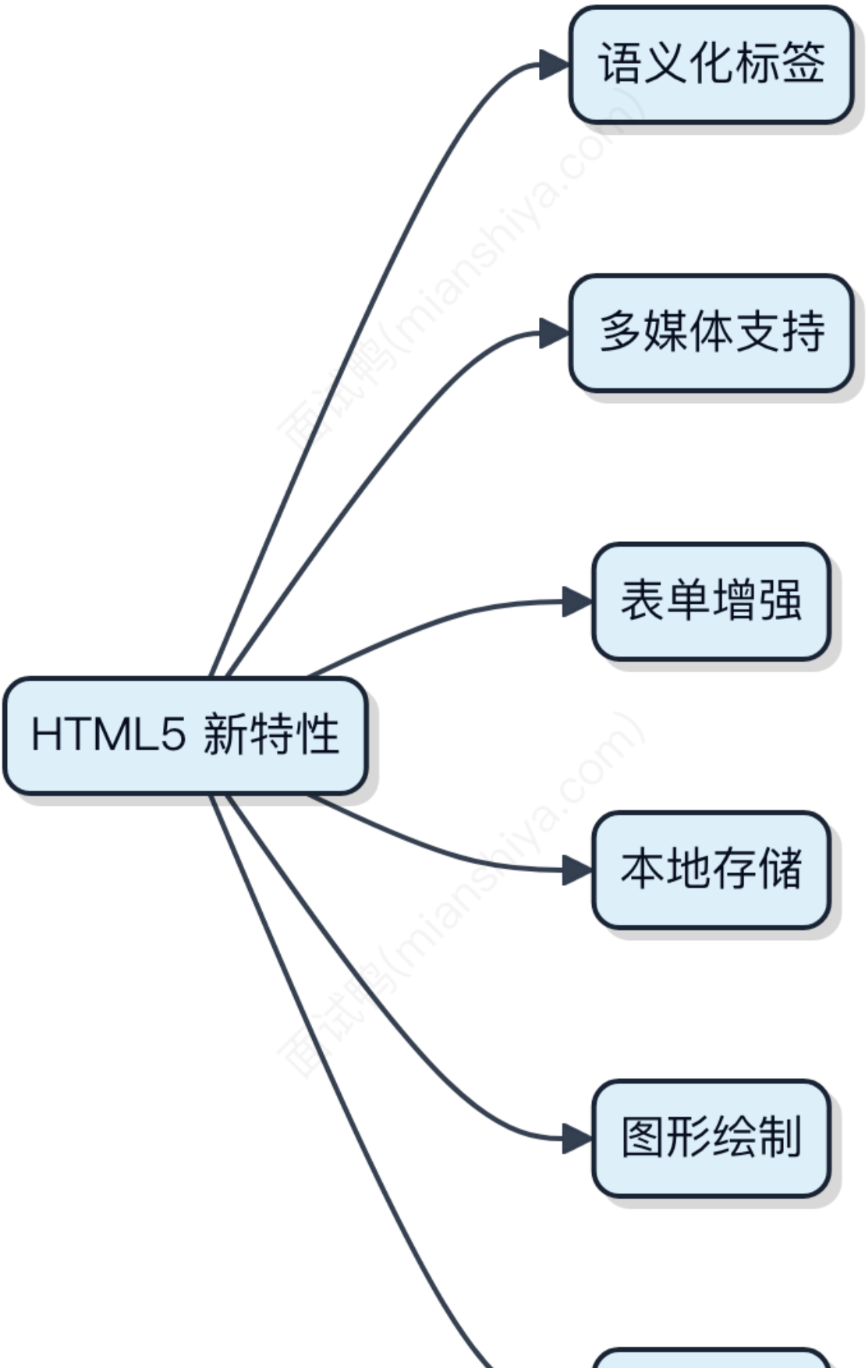
本地存储方案升级。localStorage 和 sessionStorage 替代了部分 cookie 的角色，能存 5MB 左右的数据，且不会随请求发给服务器，减轻网络负担。

离线能力通过 Application Cache 和后来的 Service Worker 推动，配合 manifest 文件或 PWA，实现离线访问和后台同步。

Canvas 和 SVG 让前端可以做图形渲染，比如数据可视化库 D3.js 就基于 SVG，游戏或图表可以直接在页面绘制。

WebSocket 实现全双工通信，和传统的轮询比，延迟更低，适合实时应用，比如聊天室用 WebSocket 能做到秒级消息到达。

面试鸭(mianshiya.com)


HTML5 新特性
语义化标签
多媒体支持
表单增强
本地存储
图形绘制

HTML 中，img 标签 srcset 属性的作用是什么？

`srcset` 属性让浏览器根据设备的屏幕密度或视口宽度，自动选择最合适的图像资源。

1）针对不同 DPR（设备像素比），可以用 `x` 描述符。比如普通屏用 1x，Retina 屏用 2x 或 3x 图。

```
<img src="logo.png"
     srcset="logo-2x.png 2x, logo-3x.png 3x"
     alt="Logo">
```

这种情况下，1x 的图会由 `src` 提供兜底，高 DPR 设备自动选更高清的版本。

2）针对不同视口宽度，用 `w` 描述符配合 `sizes` 使用。它告诉浏览器每个候选图片适用于多宽的显示场景。

```
<img src="small.jpg"
     srcset="small.jpg 480w,
             medium.jpg 800w,
             large.jpg 1200w"
     sizes="(max-width: 600px) 480px,
            (max-width: 1000px) 800px,
            1200px"
     alt="Content photo">
```

浏览器先根据当前布局算出图片的渲染宽度，再从 `srcset` 里挑最接近且大于该宽度的图。

其实这就是响应式图片的核心机制，能显著减少移动设备上的流量消耗，同时保证高清屏的显示质量。

如果原题解中的某句话使用了超链接，不要对这句话做任何改动。

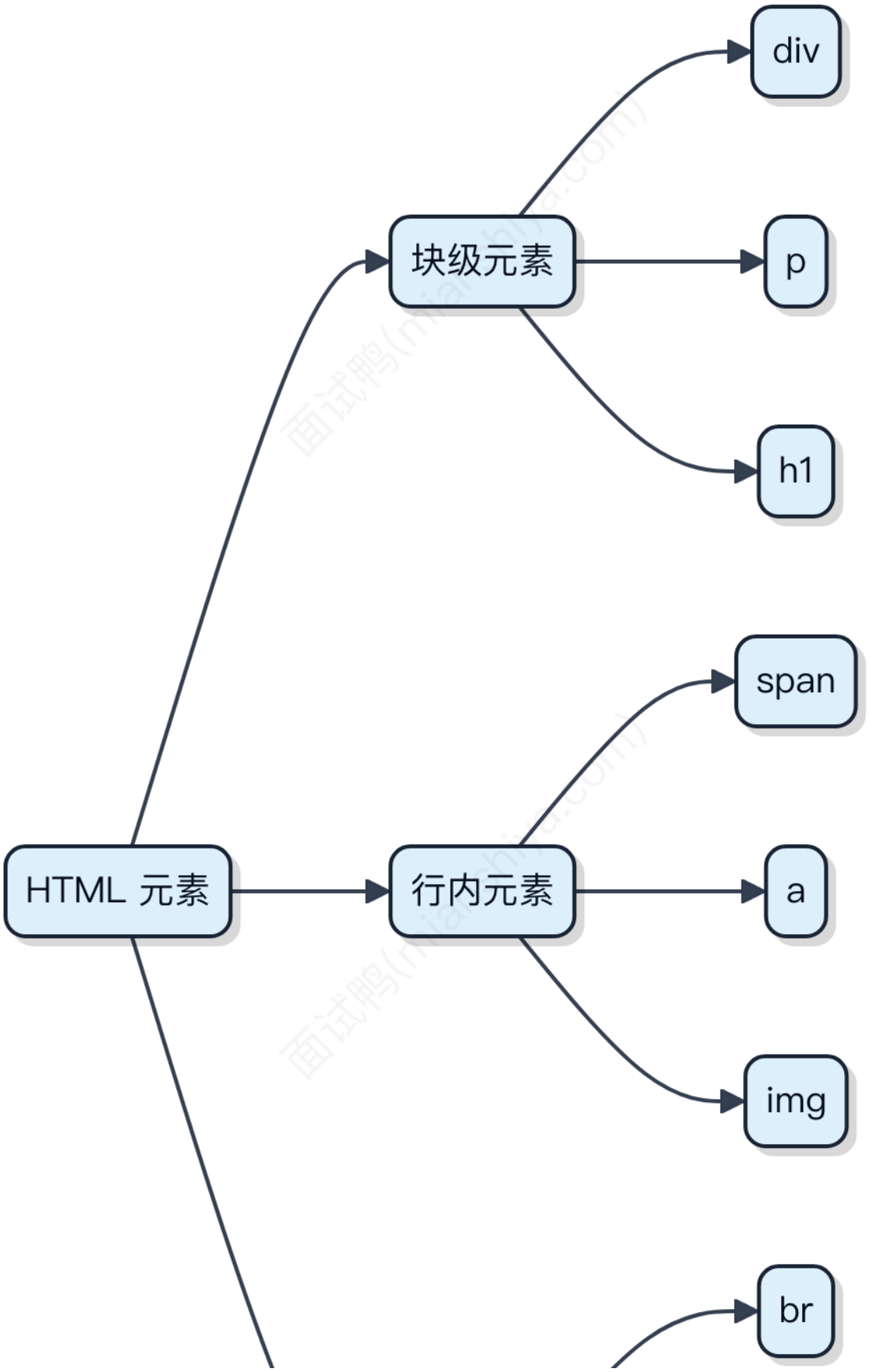
HTML 行内元素有哪些？块级元素有哪些？空（void）元素有哪些？

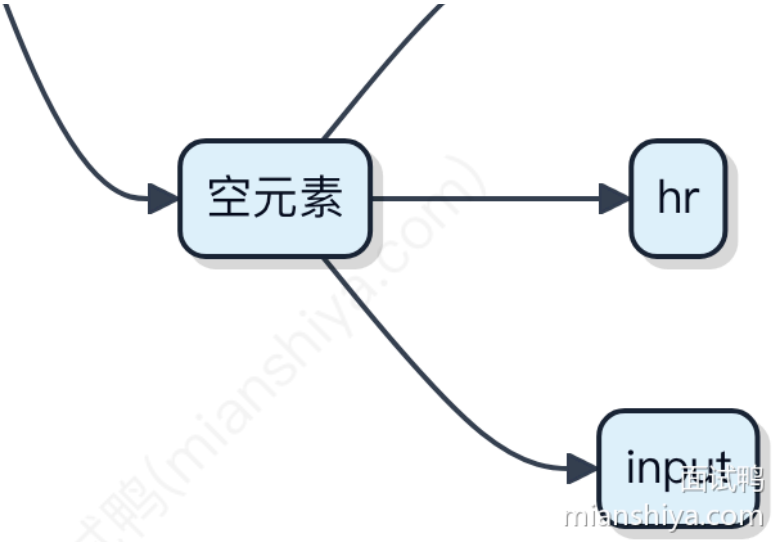
行内元素和块级元素的区别，关键看它怎么参与页面布局。块级元素默认独占一行，能设置宽高，比如 div、p、h1 到 h6 这些都是典型的块级元素。浏览器渲染时会给它们生成一个块框，从上到下排列。

行内元素不换行，多个可以并排显示，宽高由内容决定，典型代表有 span、a、strong、em、img。你给它设 width 或 height，压根不生效，得转成 inline-block 才行。

空元素也叫自闭合元素，没有闭合标签，内容为空。常见的像 br、hr、img、input、meta、link。这些标签直接结束，不能嵌套子元素。

举个实际场景：写表单时 input 是空元素，不用写 `</input>`；换行用 br 而不是多个 p 去撑间距，这就是合理利用元素特性。


HTML 元素
块级元素
div
p
h1
行内元素
span
a
img
br



面试常问“img 是行内还是空元素”，其实它两个都是。分类维度不同，不互斥。记住这个点，基本就能答全了。

HTML 的 head 标签有什么作用？其中哪些标签必不可少？

`head` 标签是 HTML 文档的元数据容器，它不直接显示内容，而是告诉浏览器页面该怎么解析、渲染和处理。

这里面放的东西，比如字符编码、页面标题、样式链接、脚本配置，都是给浏览器和搜索引擎看的“说明书”。

1） `title` 是唯一必须存在的标签。没有它，页面在标签页里没名字，搜索引擎也不知道这页是干啥的，用户体验和 SEO 都会出问题。

2） `meta charset="UTF-8"` 几乎算强制要求。如果缺了这句，中文可能乱码，浏览器也可能用错误编码解析，后续样式或脚本加载都可能受影响。

3） `meta name="viewport"` 在移动端至关重要。没有它，页面在手机上会被缩成小屏宽度，文字看不清，交互体验极差。

其他像 `link` 引入 CSS、 `script` 加载 JS、 `meta` 定义描述或关键词，属于按需使用。比如做 PWA 会加 manifest，做社交分享优化会加 Open Graph 标签。

其实现代开发中，很多框架生成的模板已经默认包含这些基础 `meta` 和 `link` ，但理解它们的作用才能在出问题时快速定位。

HTML 的 src 和 href 属性有什么区别？

`src` 和 `href` 都是 HTML 中用来引入外部资源的属性，但它们的语义和浏览器处理方式完全不同。

`src` 是 **替换** 行为。比如 `<script src="...">` 或 `<img src="...">` ，浏览器会去加载这个资源，并把它“放”到当前标签的位置上。你可以理解成：这个标签的内容就是由 `src` 指向的资源决定的，原始标签内容会被完全替换。

而 `href` 是 **链接** 行为，通常用在 `<link>` 或 `<a>` 标签里。它不会替换当前元素，而是建立一个关联。比如 `<link href="style.css" rel="stylesheet">` ，是告诉浏览器：“去加载这个 CSS 文件，然后应用到当前页面”，但它自己不展示内容。 `<a href="...">` 则是可点击的导航链接。

关键区别在于文档流的参与方式：

1） `src` 引入的资源会阻塞渲染（比如 JS），被视为页面的一部分，**直接嵌入文档树**。 2） `href` 更多是建立关联，比如样式表、图标、跳转链接，**不一定会嵌入内容主体**。

代码示例：

```
<img src="photo.jpg">    <!-- 替换为图片 -->
<link href="theme.css" rel="stylesheet"> <!-- 关联样式，不替换 -->
<a href="/home">首页</a> <!-- 导航，用户触发 -->
```

简单记：**src 是“拿来用”，href 是“挂个钩”**。

HTML 中，title 与 h1 标签的区别是什么？

title 是网页的元信息，属于文档的一部分，不会直接显示在页面可视区域内。它主要出现在浏览器标签页标题、收藏夹名称或搜索引擎结果的标题位置。比如你在百度搜索一个页面，结果中显示的大标题就是来自这个页面的 title 内容。

h1 是 HTML 文档中的结构化内容标签，是页面内真正展示给用户看的一级标题，通常表现为页面中最醒目的文字标题，一个页面一般建议只有一个 h1 来突出主要内容主题。

两者语义层级不同，作用场景也不一样。SEO 优化时，搜索引擎会同时关注 title 和 h1 的匹配度，但权重最高的还是 title。如果这两个标签表达的主题不一致，可能会影响关键词排名。

代码上写法也不同：

```
<!DOCTYPE html>
<html>
<head>
    <title>这是网页标题</title>
</head>
<body>
    <h1>这是一级标题</h1>
</body>
</html>
```

简单说，**title** 是给浏览器和搜索引擎看的，而 h1 是给访问者看的。别把它们混用，尤其别为了 SEO 堆关键词在 h1 里，反而破坏了页面结构清晰性。

HTML 中，b 与 strong 标签的区别是什么？

`<b>` 和 `<strong>` 看起来都是加粗文字，但它们的语义完全不同。

1） `<b>` 只是视觉上的加粗，不带任何语义强调。它就是告诉浏览器“这段文字要粗一点”，比如在一段文本里突出一个关键词外观，但机器和屏幕阅读器不会认为它更重要。

2） `<strong>` 表示内容的重要性或紧迫性，是**语义级强调**。搜索引擎、辅助技术（如读屏软件）会识别它，可能改变语调或优先索引。比如警告信息“删除操作不可撤销”里的“不可撤销”，就应该用 `<strong>` 。

你写页面不只是给人看，还要让机器理解。如果全用 `<b>` ，对残障用户或 SEO 就不友好。

代码上看不出区别，但语义差得远：

```
<p>这是一段普通的<b>加粗文本</b>，没有特别重要。</p>
<p>请注意：<strong>您的账户将在 24 小时后被删除</strong>，请尽快处理。</p>
```

简单说：
想改样式，用 CSS；
想表达“这很重要”，用 `<strong>` ；
`<b>` 的使用场景非常少，大多数情况你应该压根不用它。

HTML 中，i 与 em 标签的区别是什么？

`i` 标签纯粹是样式层面的倾斜展示，不带任何语义。你想让一段文字斜体就用它，但它对屏幕阅读器或者搜索引擎来说，没有任何特殊含义。

而 `em` 标签是**强调**，有明确的语义作用。它默认会表现为斜体，但这不是重点。重点是它告诉浏览器和辅助设备，“这部分内容需要被特别注意”。比如读屏软件读到 `em` 包裹的文字时，通常会加重语气。

你不能把这两个标签当作可以随意互换的“斜体工具”。如果只是装饰性倾斜，比如展示拉丁学名、外语词，可以用 `i` 。但只要你想表达“这个很重要”、“这里要强调”，就必须用 `em` 。

从 SEO 和无障碍访问（a11y）角度看， `em` 的价值远高于 `i` 。现代开发更推荐用语义化标签， `em` 是其中的基本功。

代码上它们看起来差不多：

```
<p>这是一段普通的文字，<i>这部分是斜体</i>。</p>
<p>这是一段普通的文字，<em>这部分被强调了</em>。</p>
```

但背后的意图完全不同。别图省事全用 `i` ，否则你在写的是“给机器看的文档”，而不是“给人和机器共同理解的内容”。

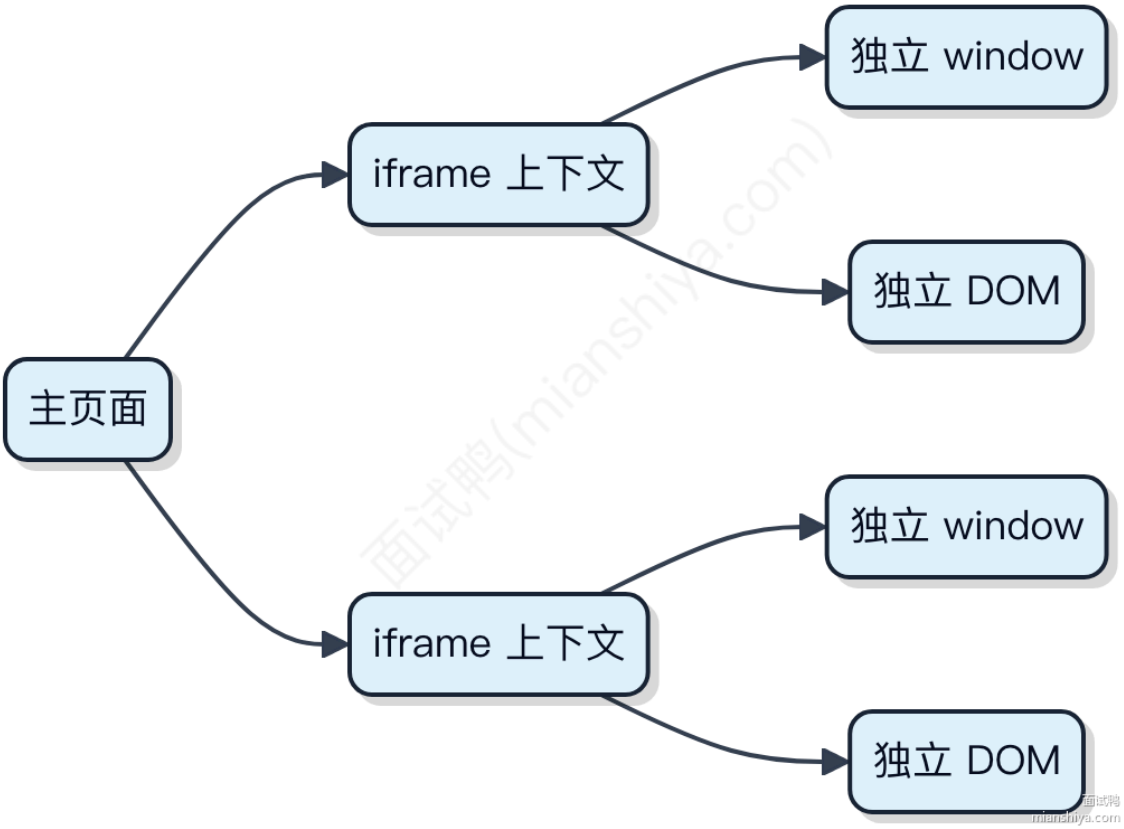
iframe 有哪些优点和缺点？

iframe 的本质是 HTML 里嵌入一个独立的浏览器上下文，它在特定场景下很有用，但也带来不少麻烦。

优点主要体现在隔离性和并行加载。页面可以嵌入第三方内容，比如广告、地图或者支付组件，这些资源能和主页面并行加载，不阻塞主线程。像 Google Maps 嵌入到企业官网时，就靠 iframe 实现内容复用和来源分离。由于有同源策略保护，不同源的 iframe 之间默认无法互相访问 DOM，这提供了一定程度的安全隔离。

缺点更值得注意。首先是性能开销大，每个 iframe 都会创建新的浏览上下文，占用额外内存和进程资源，移动端尤其敏感。其次是脚本通信复杂，跨域情况下只能依赖 postMessage，容易出错且调试困难。SEO 也受影响，搜索引擎对 iframe 内容的抓取支持不好，不利于页面收录。再者样式和响应式控制难，iframe 的高度通常需要手动设置或动态计算，父页面很难根据内容自动调整布局。

1）无法直接获取跨域子页面的尺寸，导致滚动条嵌套 2）主页面的前进后退会受 iframe 历史记录影响 3）移动端 Safari 对 iframe 的某些 API 支持有限



现在主流方案更倾向用 Web Components 或微前端架构替代 iframe 来实现模块隔离，既保留解耦优势，又避免了上下文切换的代价。

HTML 中，label 标签的作用是什么？如何使用？

label 标签的主要作用是提升表单的可访问性，让点击文字也能触发关联的表单控件，比如 checkbox 或 input。这对鼠标用户和辅助设备都更友好。

使用方式有两种。第一种是通过 `for` 属性绑定目标元素的 id：

```
<label for="username">用户名</label>
<input type="text" id="username">
```

点击“用户名”文字时，光标会自动聚焦到输入框。

第二种是嵌套写法，把控件直接放在 label 标签内部，这种不用写 for 和 id：

```
<label>
  <input type="checkbox" name="agree"> 同意协议
</label>
```

这种方式更简洁，不容易出错，适合简单的表单项。

注意点：每个 label 只应关联一个表单元素，重复绑定会导致行为混乱。如果是 radio 组，每一个 radio 都要有自己的 label 或嵌套结构。

实际开发中，像 Ant Design、Element Plus 这些组件库，内部都默认用 label 包装了 checkbox 和 radio，你可能没明显感知，但它一直在起作用。

浏览器是如何对 HTML5 的离线储存资源进行管理和加载的？

HTML5 的离线存储主要靠 **Application Cache**（简称 AppCache），不过得先说一句，这玩意儿已经被废弃了，现代项目别再用了。但面试常问，还是得搞明白它咋工作的。

浏览器加载页面时，会先检查页面是否有 `<html manifest="cache.manifest">` 这个属性。有就去请求这个 manifest 文件，里面列了三类资源：

1）CACHE：必须缓存的文件，比如主 JS、CSS
2）NETWORK：每次都要走网络的，比如 API 接口
3）FALLBACK：网络失败时的备用页，比如 /api /offline.html

第一次访问，浏览器把 CACHE 列表里的资源全拉下来存本地。之后再进这个页面，**直接从本地加载**，压根不经过服务器，除非 manifest 文件内容变了——注意是内容变，不是时间戳改。

有个坑特别多：只要 manifest 里任何一个资源下载失败，整个缓存更新就失败。而且用户看不到进展，页面可能卡死不动。

其实现在应该用 **Service Worker + Cache API** 做离线存储，控制粒度更细，还能拦截请求。AppCache 太粗暴，连 Chrome 都把它干掉了。

Canvas 和 SVG 有什么区别？

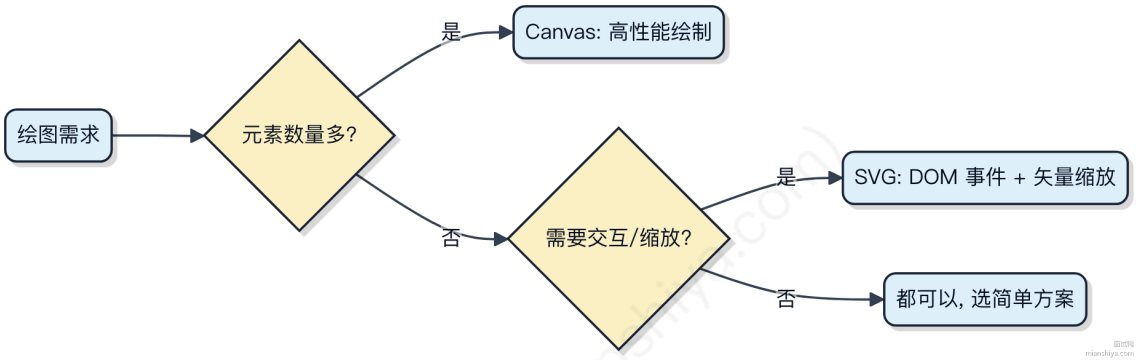
Canvas 和 SVG 都是前端绘图技术，但设计思路完全不同。

Canvas 是基于位图的即时模式渲染。你拿到一个画布，用 JavaScript 一个像素一个像素去画，画完就不管了。浏览器不记录你画了什么，改个颜色得重绘整片区域。适合高频率绘制的场景，比如游戏、实时视频处理、数据可视化大屏，像 ECharts 在大量数据点时就会切到 Canvas 模式。

SVG 是基于矢量的保留模式渲染。它本质是 DOM 结构，每个图形都是一个 XML 元素，比如 `<circle>`、`<rect>`。你可以给它们绑定事件、做 CSS 动画，浏览器会记住每个对象的状态。缩放不失真，特别适合图标、流程图、交互式图表这类需要精细操作的场景。

性能上，Canvas 在画几千个元素时依然流畅，而 SVG 的 DOM 节点多了之后，内存和重排开销会明显上升。反过来，SVG 对小规模、高交互的内容更轻量，不需要手动管理重绘逻辑。

1）Canvas 控制的是像素，适合密集绘制
2）SVG 控制的是对象，适合结构化图形
3）Canvas 无法通过 DOM 获取图形，SVG 可以



选哪个，关键看是不是要“记住”图形本身。要记，上 SVG；要快，上 Canvas。

HTML5 的离线储存怎么使用？它的工作原理是什么？

HTML5 的离线储存主要靠 **Application Cache**（简称 AppCache），不过得先说清楚，这玩意儿早就被废弃了，现在正经项目里压根不用。面试问到更多是考察你是否了解技术演进。

真正靠谱的方案是 **Service Worker** 配合 Cache API 和 IndexedDB。流程是这样的：页面首次加载时，注册一个 Service Worker，它会在后台偷偷缓存关键资源。下次用户没网时，请求直接被 Service Worker 拦截，从缓存里拿数据，页面照样能打开。

1）先在主进程中注册 Service Worker

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('/sw.js');
  });
}
```

2）在 `sw.js` 里监听 fetch 事件做缓存策略

```
self.addEventListener('fetch', (e) => {
  e.respondWith(
    caches.match(e.request).then((r) => r || fetch(e.request))
  );
});
```

这套机制是现代 PWA 的基础，像 Twitter Lite、Google Maps Go 都这么干。优势在于完全可控，可以精确指定哪些接口走网络、哪些用缓存，还能后台同步数据。

AppCache 的问题就是太粗暴，缓存更新不灵活，容易导致用户卡在旧版本，而且和浏览器默认缓存策略打架。所以别踩这个坑，记住用 Service Worker 才是正解。

什么是 HTML 语义化？

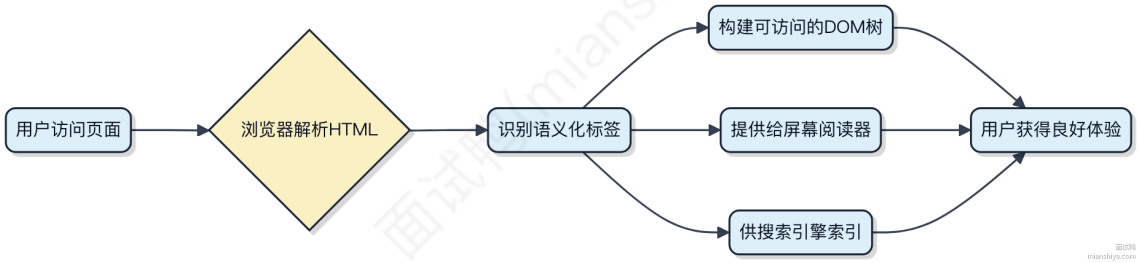
HTML 语义化指的是使用具有明确含义的标签来构建页面结构，而不是用一堆 div 堆出来。每个标签都代表特定内容类型，比如 `article` 表示独立文章，`nav` 用于导航区块，`footer` 放页脚信息。

浏览器和辅助设备能更好理解页面结构，搜索引擎也能更准确抓取内容。像屏幕阅读器靠这些标签帮助视障用户快速定位，如果全用 div，那整个页面就跟天书一样。

对开发者来说，代码可读性也强得多。看到 `main` 就知道是主体内容，看到 `aside` 就明白是侧边栏，维护起来省心不少。

常见的语义化标签有 `header` 、 `section` 、 `article` 、 `figure` 、 `figcaption` 、 `time` 等。比如写一篇博客，用 `article` 包裹标题和内容，里面再用 `header` 放标题， `footer` 标发布时间，结构清晰。

1）避免滥用 div 和 span 2）根据内容意图选标签，不只看默认样式 3）配合 ARIA 属性增强复杂组件的语义表达



> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*